

A Database for the Marketing Department

Student ID: 12313216, 12310908
Name: 林鸿渝/Linhy, 马彦昭/Mayz
Course: CS307: Principles of Database Systems
Lab: 2-78 Zhu
Date: 2025 年 4 月 28 日

目录

1	任务划分及贡献	2
2	实体-关系图绘制	2
3	数据库设计	3
3.1	Datagrip 生成图	3
3.2	数据库设计简述	3
3.2.1	数据描述	3
3.2.2	属性划分	4
3.2.3	约束条件	4
4	数据导入	5
4.1	数据导入说明	5
4.1.1	数据修改说明	5
4.1.2	脚本使用说明	5
4.2	导入数据准确性验证	6
4.2.1	postgresql 查询语句	6
4.3	导入方式拓展	7
4.3.1	多语言导入: Python	7
4.3.2	导入优化	8
4.3.3	Java 实现的另一导入方式	10
4.3.4	多平台导入	11
4.3.5	不同导入数据量对于性能的影响	12
4.3.6	trigger 对于数据导入的影响	12

1 任务划分及贡献

成员	任务	贡献比
林鸿渝	ER 图绘制、Python 实现数据导入、不同平台性能测试、 trigger 对于性能的影响	50%
马彦昭	Java 实现数据导入、Java 实现多方法 报告书写、构建建表语句、方法优化、 导入数据量对于性能的影响	50%
*		*

2 实体-关系图绘制

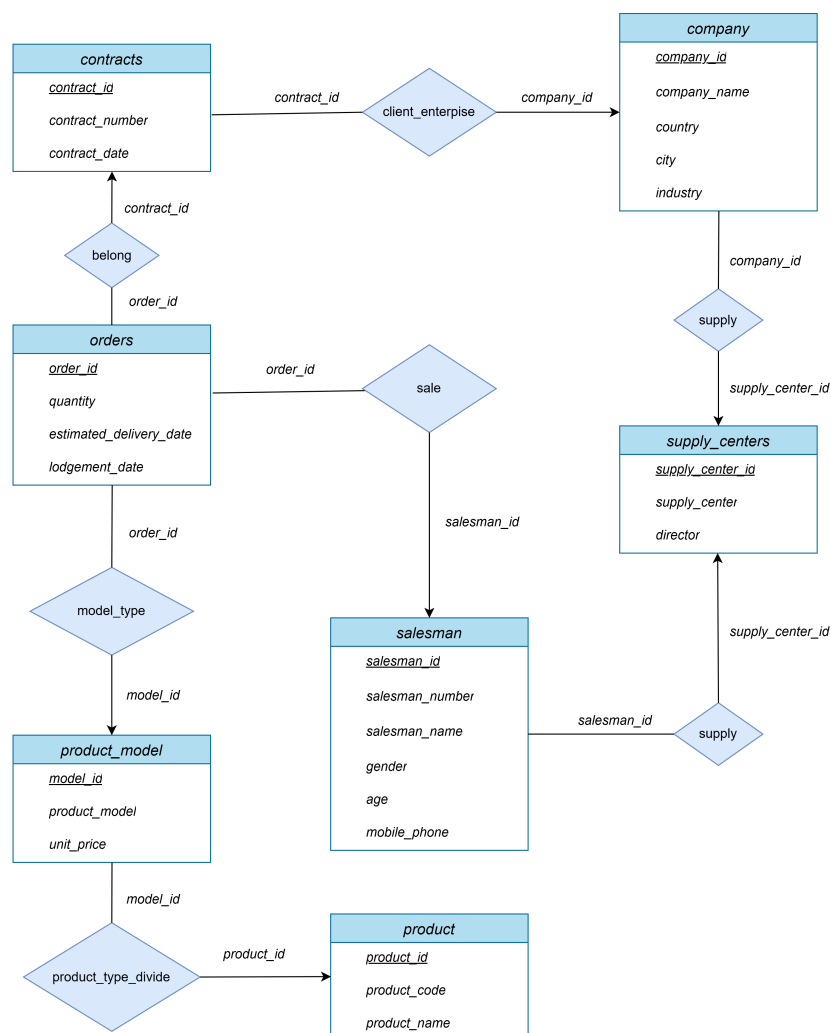


图 1: ER 图

本 ER 图通过“draw.io”工具辅助绘制 (<https://www.drawio.com/>)

3 数据库设计

3.1 Datagrip 生成图

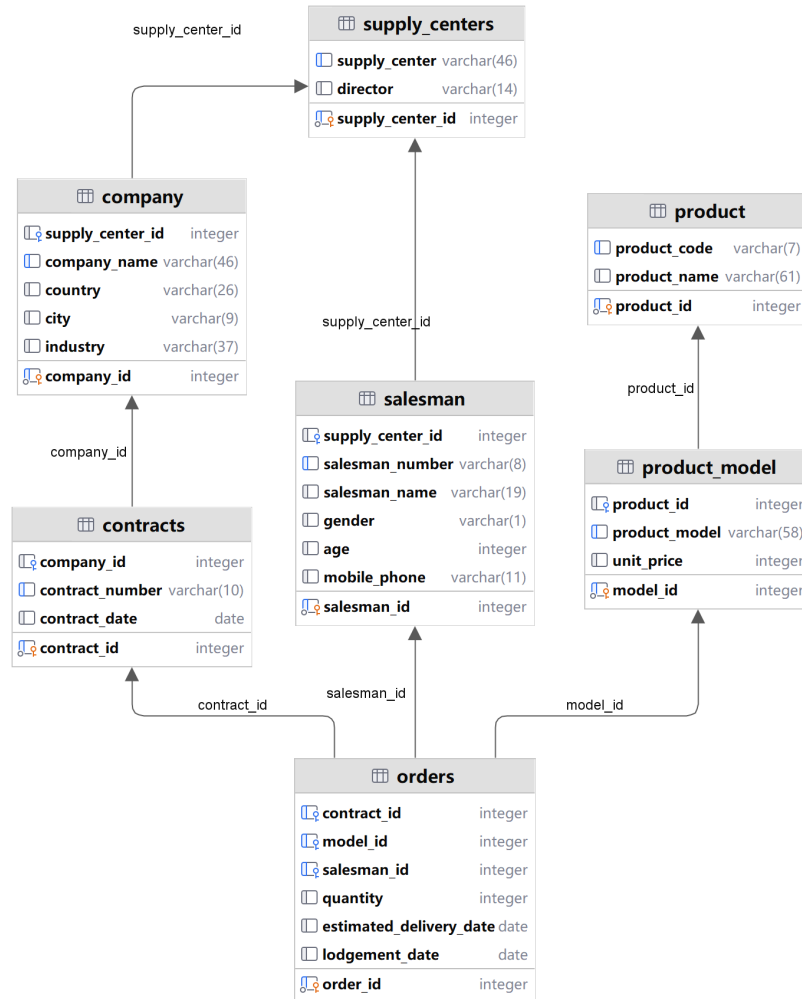


图 2: Datagrip 生成图

3.2 数据库设计简述

3.2.1 数据描述

本数据库以 Output25S.csv 为基础建立。在原始的 csv 文件中，包含了 500,000 条数据，即 500,000 条订单。任意订单均包含合同编号 (contract_number)、客户公司 (client_enterprise)、公司所在国家 (country)、城市 (city)、所属行业 (industry)、产品编号 (product_code)、产品名称 (product_name)、产品型号 (product_model)、产品型号对应单价 (unit_price)、订单产品数量 (quantity)、合同下单日期 (contract_date)、产品预计到达日期 (estimated_delivery_date)、实际到达日期 (lodgement_date)、公司所在区域管理员 (director)、公司所在区域的供应中心 (supply_center)、销售员 (salesman)、销售员编号 (salesman_number)、性别 (gender)、年龄 (age)、电话 (mobile_phone)，共计 20 类属性。

其中，若客户公司不在中国，其所在城市 (city) 对应值可为 NULL；若实际送达日期 (lodgement_date) 在 2025-3-24 之后，则该值也可 NULL。

导入数据的类型已在生成图中展示，报告空间有限，不再赘述。

3.2.2 属性划分

1NF: 所有字段均为原子值。Output25S.csv 中所有属性均为原子属性，且没有重复的列，因此满足 1NF。

2NF: 非主键字段完全依赖于主键。由此将原表格划分为以下子表：

子表	属性
订单	<u>订单 ID</u> 、订单产品数量、预计到达日期、实际到达时期
合同	<u>合同 ID</u> 、 <u>合同编号</u> 、合同签署时间
公司	<u>公司 ID</u> 、 <u>公司名</u> 、公司所在国家、城市、所属行业
销售员	<u>销售员 ID</u> 、 <u>销售员编号</u> 、姓名、性别、年龄、电话
产品型号	<u>产品型号 ID</u> 、 <u>产品型号</u> 、型号单价
产品	<u>产品 ID</u> 、 <u>产品编号</u> 、产品名称
供应中心	<u>供应中心 ID</u> 、 <u>供应中心名称</u> 、所在区域管理员

3NF: 非主键字段中不能有传递依赖。检查上表，所有子表的元素都直接与主键 (ID) 相关，所有非主键字段都直接依赖于主键，不是通过其他非主键字段间接依赖。

3.2.3 约束条件

主键约束: 所有子表主键均为子表对应 ID，已在上表中以下划线标注。

外键约束: 3.2.2 属性划分下的表中不包含外键属性。在此部分加入。定义一对多关系: $R(1,x)$ ，我们可找到如下表间关系: (supply_centers, company), (supply_centers, salesman), (company, contracts), (product, product_model), (contracts, orders), (salesman, orders), (product_model, orders)。以此一对多关系连接不同子表，足够实现完整数据库，无需创建中间表声明表间关系。

子表	外键 (指向表)
订单	<u>合同 ID</u> (contracts)、 <u>销售员 ID</u> (salesman)、 <u>产品型号 ID</u> (product_model)
合同	公司 ID(company)
公司	供应中心 ID(supply_center)
销售员	供应中心 ID(supply_center)
产品型号	产品 ID(product)

唯一约束: 所有表格均有唯一约束。除去 orders 表，其余子表 Unique 约束均已在表 2 中以加点形式标明。orders 表以三个外键作为联合 Unique 约束，已在上表以加点形式标注。

4 数据导入

4.1 数据导入说明

4.1.1 数据修改说明

原 csv 文件中, 存在错误数据使得同一销售员编号对应不同销售员。

s1.id	s1.name	s1.number	s1.gender	s1.age	s1.mobile_phone
2120	Zhao Rongchun	12030116	Male	46	15074922863
2202	Ralph Thompson	12435613	Male	44	18998586749
2673	Patrick Johnson	12030116	Male	38	15059073373
2693	Alistair Walker	12435613	Male	47	15066175538

图 3: 通过 Cross join 找到的错误数据

进行数据库构建时, 我们对所有含有上述字段的数据进行如下修改:

```
... Patrick Johnson 12030116 ... → ... Patrick Johnson 12313216 ...
... Ralph Thompson 12435613 ... → ... Ralph Thompson 12310908 ...
```

4.1.2 脚本使用说明

Script Name	Author	Description
App.java	Mayz	运行该程序, 先初始化数据库与 Reader, 再创建与表一一对应的语句, 对每个语句单独计算缓冲大小, 逐行完成数据载入
DBConnection.java	Mayz	读取配置文件并初始化数据库连接
StatementPreparer.java	Mayz	对语句使用提供基本支持, 如使用 JDBC 的缓冲方法、对不同的目标数据类型自动选择处理方式
InsertMethods.java	Mayz	静态类, 完成对从文件中读取数据的处理, 如去重, 添加 ID 等, 将其整合后注入 insert 语句中
MyUsedSQLs.java	Mayz	枚举类, 存储 sql 语句, 方便改, 查
UsedDataType.java	Mayz	枚举类, 用于作为参数传递需要填入表中的数据类型

具体导入流程:

1. 读取配置文件, 连接数据库, 读取原始数据文件
2. 执行清理建表语句, 创建不同表的 PreparedStatement 对象, 并用数组存储 PreparedStatement 对象。
3. 使用 while 循环, 逐行读取 csv 文件, 按照外键依赖, 顺序执行 batch 条件判断

4. 结束循环，最后执行一次提交语句，确保不漏，关闭 reader 和连接，计时

4.2 导入数据准确性验证

4.2.1 postgresQL 查询语句

```
-- 1. 不同性别的销售员分别有多少个
select distinct gender, count(distinct salesman_name)
from salesman
group by gender;

-- 2. 每一个supply center中有多少公司
select supply_center, count(distinct company_name)
from company
      join supply_centers on company.supply_center_id =
      supply_centers.supply_center_id
group by supply_center
order by supply_center ASC;

-- 3. 每一个supply center中有多少销售员
select s.supply_center, count(distinct salesman_id)
from supply_centers s
      join salesman m on s.supply_center_id = m.supply_center_id
group by s.supply_center
order by supply_center ASC;

-- 4. 年龄在某个上界或下界区间的销售员有多少个
select count(distinct salesman)
from salesman
where age > 42 and age < 45;

-- 5. 每一个supply center中有多少国家
select s.supply_center, count(distinct country)
from company c
      join supply_centers s on c.supply_center_id = s.supply_center_id
group by s.supply_center
order by supply_center ASC;

-- 6. 根据某一个国家名称，列出其所有的公司名称，及其公司所属行业
select distinct country, company_name, industry
from company
where country = 'Italy'
order by country, company_name;
```

```

-- 7. 根据某一个产品编码，列出其所有的产品型号、及对应的单价
select distinct p.product_code, m.product_model, m.unit_price
from product p
      join product_model m on p.product_id = m.product_id
where product_code = 'B56K283'
order by p.product_code, m.product_model, m.unit_price;

-- 8. 根据某一个合同编码，列出合同中所有订单内容，
每个订单包含（产品型号、购买数量、销售员名称、实际到达日期）
select distinct c.contract_number, p.product_code, m.product_model, o.quantity,
      s.salesman_name, o.lodgement_date
from contracts c
      join orders o on c.contract_id = o.contract_id
      join salesman s on o.salesman_id = s.salesman_id
      join product_model m on o.model_id = m.model_id
      join product p on m.product_id = p.product_id
where contract_number = 'CSE0000267'
order by c.contract_number, m.product_model, o.lodgement_date, s.salesman_name,
      o.lodgement_date;

```

4.3 导入方式拓展

本拓展的环境及配置均详见 4.3.4 中的 Window 系统栏目

4.3.1 多语言导入：Python

Script Name	Author	Description
LoadInStages.py	Linhy	运行该 py 文件完成数据导入
DbConnection.py	Linhy	读取配置文件进行数据库连接与关闭，获取 cursor 及 con
InsertMethod.py	Linhy	结合不同 sql 的 PreparedStatement 对象，提供不同 Insert 语句对应的插入方法。与此同时维护一个字典 (Hash)，便于之后添加不同表内的主键 id
PreparedStatement.py	Linhy	每个语句对应一个 pstmt，维护 sql 语句以及参数数组
DataType.py	Linhy	枚举类，作为 pstmt 传入参数的类型选择，进行不同 if 语句
SqlSentences.py	Linhy	不同 sql 语句，包括建表删表以及插入数据、增加约束

具体导入流程：

1. pandas 读 csv 文件，并进行预处理，删除最后一行以及将 “ ” 和 nan 转为 None。

2. 读取配置文件，连接数据库，获取 stmt(cursor) 及 con。
3. 给所有 sql 语句，包括建表删表以及插入数据、增加约束，创建 PreparedStatement 对象。
4. 删除所有已有的表，创建数据库设计的 7 个表格，create 语句中已添加约束。为下面插入数据做初始准备。
5. 逐行读取 csv 文件，先向 supply_center 以及 product 这两个完全没有外键约束的表格插入数据。具体实现为：调用 InsertMethod 的插入方法，存至对应 pstmt 的参数数组，当执行数量达到 batch 便执行 executemany。commit 提交后 clear 参数数组。最后统一再 commit 一次，处理余数。
6. 同样的实现逻辑，依次插入有上述两张表的外键依赖的：salesman、product_model、company。再插入依赖 company 的 contracts。最后插入 orders。

Python 实现的方式与 Java 有所不同，Java 是把一行数据拆分到每条指令，处理后先 execute 放入缓存区，在统一 commit。而 Python 是把每个数据分了四个阶段分别放入缓存区，且 sql 语句之间不再允许交叉。原始的 Java 代码在每次插入数据均调用 executeUpdate，非常低效，本 Python 实现使用 executemany 实现批量导入，减少与数据库间的访问次数，与 Java 第二次优化使用的 addBatch 类似。下面与 Java 第二次优化的导入性能做对比：

Java 第二次优化：25181 records/s Python 原始：11055 records/s

我们发现，Java 的 addBatch 竟然比 Python 的 executemany 快了近一倍?! 查阅资料发现 Python 调用的 pycpg2 的 executemany 是伪批量操作，实际是循环插入，性能与 execute 接近。对比 Java 原始的效率，为 13378 records/s，与 Python 接近，再加上 Python 的运行时开销比 Java 大，这样的结果符合预期。

4.3.2 导入优化

- **Java:**

初始版本：参照基础的方法，在 InsertMethods.java 中使用 hashmap 将主键和生成的 ID 对应并完成去重，每次缓冲区满统一执行。

第一次优化：在 InsertMethods.java 将主键优先提取，避免重复提取主键的操作；在 App.java 中调用 InsertMethods 中下下次要赋的 ID 作为数据量，将各语句分别计算缓冲。

第二次优化：经过进一步学习 JDBC，在 StatementPreparer.java 中将原先的 update 方法改为 addBatch 方法，并在 App.java 中加入 executingXXX 方法，方便嵌套调用，避免出现执行 batch 时相关语句未提交的错误。

第三次优化：进一步优化了分别缓冲的逻辑，在 StatementPreparer.java 中加入了一个 int 数据标记 batch 中的数据量，加入了 batch 为空的逻辑判断语句，减少了重复执行空 batch 的次数。

从“(a)Java 优化”可得，第一次优化改变了将原始数据的记录改变为目标表中记录的逻辑，

变为统一处理主键，通过各语句分别缓冲，减少了提交代码的次数，得到了大约 14% 的效率优化；第二次优化替换 JDBC 执行方法，利用 PreparedStatement 的批量操作特性并引入嵌套提交逻辑，消除没有对应外键的约束错误，得到了大约 65% 的效率优化；第三次优化改变提交语句的判断条件，更严格的限制了执行，减少了不必要的执行，避免了执行空 batch，得到了大约 92% 的效率优化；总体而言，Java 优化主要聚焦于数据库交互层面的深度重构。前两次优化通过重构主键处理流程和 JDBC 批处理机制，实现了事务级的性能提升。第三次优化通过引入批量标记与空操作检测机制，精准控制数据库写入流程，消除空批处理的性能开销。最终实现了约 3 倍的性能提升。

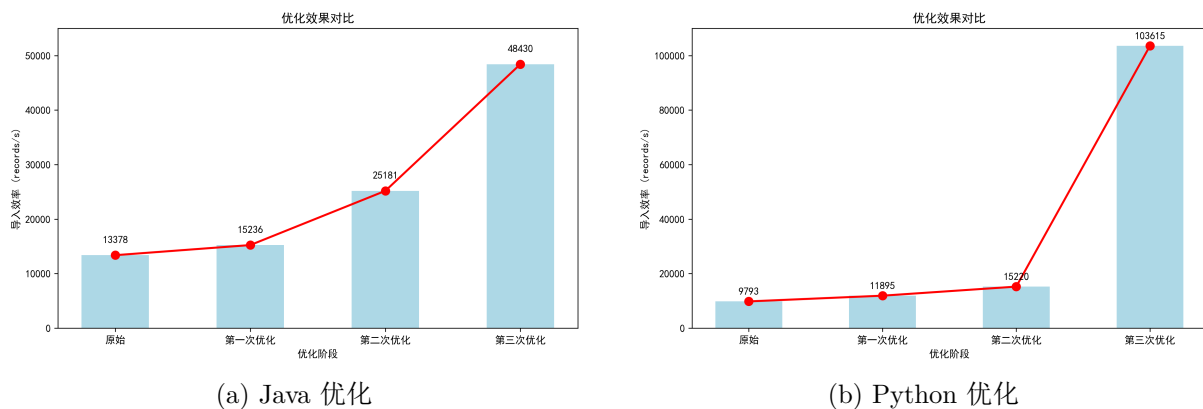


图 4: Java 与 Python 优化对比

• Python:

初始版本：先导入 supply_center, product 两张无外键的独立表，再导入 salesman, product_model, company 三张存在关于上一张表的外键约束的表格，随后导入依赖 company 的 contracts，最后导入 orders。

第一次优化：初始版本建表时便添加外键及唯一约束，分四个阶段导入，每次都需重新读取数据，以及检查约束条件。本方法采取先建表，不进行任何约束，最后统一添加外键与唯一约束。

第二次优化：直至第一次优化，都需要用到 PreparedStatement.py 文件，每次都要对不同的 sql 语句创建 pstmt 对象，并进行访问，较为耗时。将其删去，对 InsertMethod 进行修改，将其内部方法的 pstmt 变量改为 list，将要修改的变量存入 list，而不是 pstmt.list。

第三次优化：上述三个导入方式均采用 psycopg2.cursor.executemany() 来导入，现换为 copy 方法，通过 copy_expert 和 StringIO 将 pandas 数据帧写入数据库。

从“(b)Python 优化”可得：第一次优化延迟了约束检查，减少了约束验证开销，有 21.4% 的提升，效果明显；第二次优化移除了 PSTMT，使用 list 直接存储 SQL 要修改的变量，减少 Python 层面的计算和内存开销，提升了 32.4% 的性能；第三次优化使用 copy 命令，从根本上消除了逐条插入的开销，如 SQL 解析、网络往返，插入效率提升 580.9%。总体而言，前两次优化基于 Python 层面，提升有限，而第三次优化的 copy 命令在数据库层面，直接将数据流式写入 PostgreSQL 的存储层，绕过 SQL 解析和逐条事务写入，数据加载后再统一处理索引和约

束，开销极低。除此之外，使用顺序 I/O 而不是随机 I/O，即数据按顺序写入文件的末尾，形成连续的数据块。这种写入模式，磁盘只需要在文件末尾连续追加数据，无需频繁跳转到不同的磁盘位置。

4.3.3 Java 实现的另一导入方式

Script Name	Author	Description
GenCSVToLoad.java	Mayz	运行该文件，载入原始数据后输出加入 ID 的中间表，中间表被直接不去重输入，得到重复表，使用 SQL 语句从重复表创建最终表
GenID.java	Mayz	根据主键，使用 hashmap 去重，生成 ID
DBConnection.java, StatementPre- parer.java, MyUsedSQLs.java	Mayz	复用部分

具体导入流程：

1. 读取配置文件，连接数据库，完成初始化。
2. 逐行遍历，对每一个记录单独提取主键生成 ID，并生成加入 ID 的新记录，使用 Buffered-Writer 缓冲后写入，生成中间表
3. 逐行读取中间表，将中间表一条记录拆分成不同表的不同记录，使用单一 n 计数，到达 batch 后统一写入，生成七张未经去重的重复表
4. 创建最终表，使用 SELECT, DISTINCT 去重后写入，得到最终表，移除重复表，关闭连接，计时

Java: 27967 records/s

从过程中可得，这一方法总共需要遍历三次，第一次遍历读取原文件，生成中间表；第二次遍历读取中间表，将中间表中的数据重复地填入数据库中，生成重复表；第三次遍历在数据内部进行，筛选出不重复的数据，得到最终表。由于需要经历三次遍历，这一方法可以预想地，相较于边运行边去重，只遍历一次的第一种方法效率较低，且由于第二次遍历时不对数据进行筛选，重复填入，需要执行的语句数量也较优化后的第一种多上数倍，这些因素综合造成了该方法效率较低。在时间消耗上，对于 50 万条数据的原始数据，第一次遍历需要大致 1.6 秒，第二次遍历需要大致 16 秒，第三次遍历需要大致 1.5 秒。可见主要的时间消耗来源于大量访问数据库，而虽然数据量上基本等同，但得益于 CSVReader, CSVWriter 和 postgresQL 的优化，第一，二次遍历速度较快。但该方法也有部分可取之处，使得其效率能够达到 27000 条每秒的水平，这主要得益于该方法逻辑较为简单，需要执行的判断较少，中间变量较少，将大量判断交由 postgresQL

完成，借助 postgresQL 的优化提高了自身的效率。但也正是这一思路，使得其无法优化第二次遍历的效率，导致性能受限。

4.3.4 多平台导入

本小节测试在 Windows 和 MacOS 上运行的性能差异。其中配置环境如下：

系统	配置	运行环境
Windows	intel i9-13900HX、16GB RAM	PostgreSQL 17.4、Python 3.11、Pandas 2.0.3、Pycopg2 2.9.9
MacOS	Mac M2 Max、32GB RAM	PostgreSQL 16.2、Python 3.11、Pandas 2.1.4、Pycopg2 2.9.10

我们在不同平台上运行了相同的导入脚本，并记录了每个平台的运行时间。以下是测试结果：

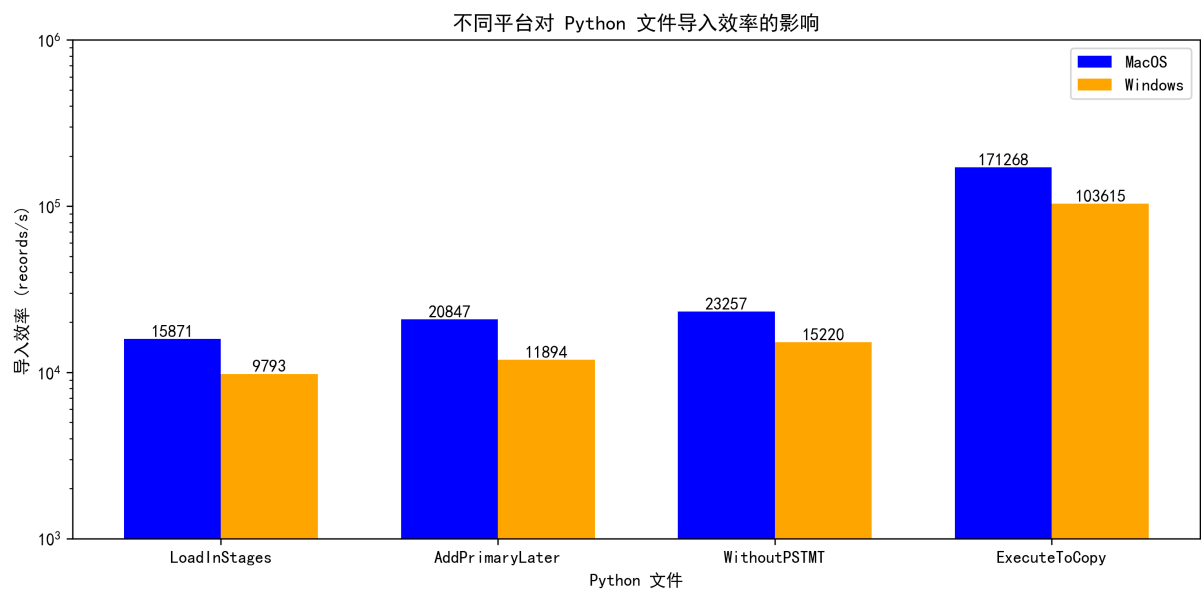


图 5: 不同导入平台对于性能的影响

比对结果可得，MacOS 的性能明显优于 Windows，主要原因在于 MacOS 的硬件优势，其内存带宽 (400GB/s) 显著大于 i9-13900HX(89GB/s)，使得数据传输速度更快。而处理器层面的 M2 Max 的高效单核性能，以及 macOS 对于 Apple Silicon 的优化，使得在数据处理和数据库交互方面表现更佳。此外，macOS 的文件系统和内存管理机制也可能对性能产生积极影响。但在这个测试中，硬件配置的影响已经占据了主要因素，在对比不同平台对于导入性能的影响层面，意义并不显著。考虑这一点拓展应该是关注不同平台对于导入性能的影响，或许在以后的深度研究中，我们可以进一步测试在硬件配置类似的情况下，操作系统对于导入性能的影响。得到的结论应该就是，macOS 对于 Apple Silicon 的优化，使得在数据处理和数据库交互方面表现更佳，这个前文已经提及。

4.3.5 不同导入数据量对于性能的影响

此部分测试不同数据量的数据导入，对于整体导入性能或者效率的影响。我们决定基于给定的 Output25S.csv，取其子表，数据量分别为 25 万、12.5 万、6.25 万、3.125 万、1.5625 万，分别进行数据导入测试。我们使用了与上面相同的脚本，并记录了每个数据量的运行时间。

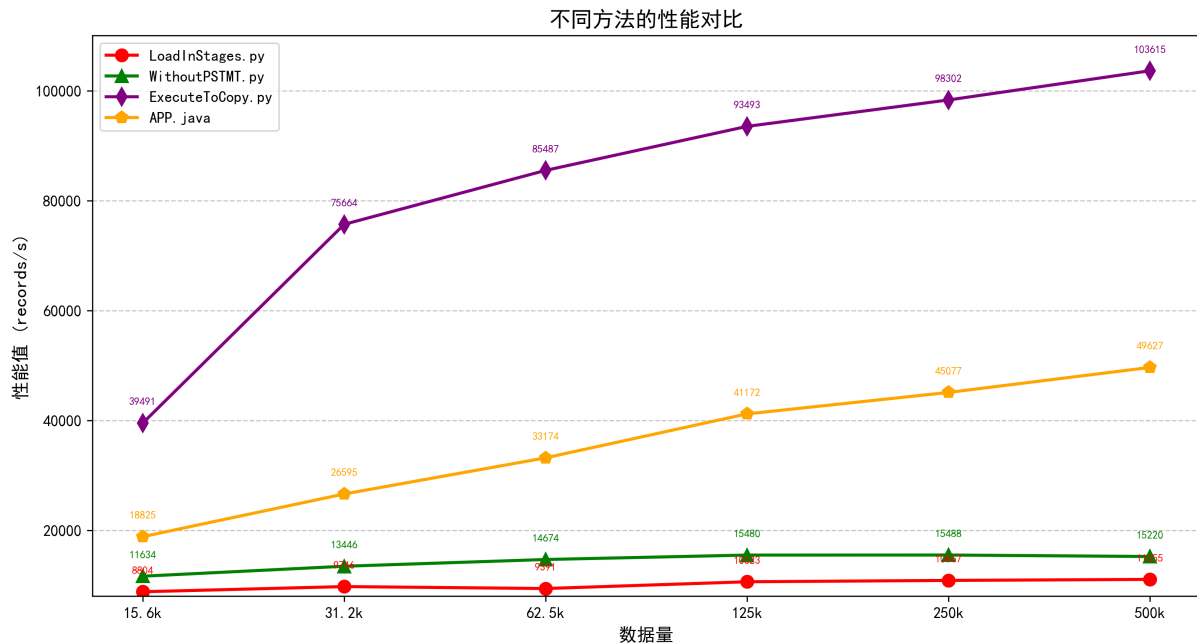


图 6: 不同数据量对于导入性能的影响

总体趋势而言，所有方法在 15625 的数据量下的性能均处于最低值，而随着数据量的增加，不同方法的导入性能均有所提升。考虑连接数据库与创建哈希 Map 等固定开销在数据量较小的情况下会有较大占比，导致性能偏低。而随着数据量的提升，固定开销的权重被插入数据稀释，使得性能呈上升趋势。

除此之外，可以发现第三次优化后的 App.java 以及 ExecuteToCopy.py 的性能提升最为显著。原因在于 App 使用的优化 JDBC 批量处理，以及 ExecuteToCopy 使用的 copy 方法下的单条记录插入时间极低，显著小于其他方法的单条记录插入时间。而随着数据量的提升，权重更多分配与导入速度，于是增幅最大，上升最为显著。具体细节已在 4.3.2 优化部分进行说明。

4.3.6 trigger 对于数据导入的影响

PostgreSQL 在 Insert 时会触发默认的约束检查，属于内部触发器。现在用 Disable trigger all 的 sql 语句进行触发器禁用。其实实现方式与 Python 第一次优化类似，但是当时没有取消主键依赖，现在可以进一步禁用主键约束检查的触发器。对 LoadInStages.py 进行修改，添加禁用触发器的 SQL 语句，测试性能差异。

able: 9793 records/s

disable: 12290 records/s

性能提升了 25.5%，说明此方法能有效提升性能。但是这样插入数据易导致数据传入错误，极其不安全。本项目的 Python 优化是先导入再检查约束，均没有爆出异常，说明数据安全传入。